Specification-Driven Development

Specification-Driven Development

How to make AI implementation reliable by treating specifications, not code, as the asset that lasts

[AUTHOR NAME]

Self-published draft

Copyright © 2026 [AUTHOR NAME].

All rights reserved. No part of this book may be reproduced, distributed, or transmitted in any form without prior written permission from the author, except for brief quotations used in a review.

Draft edition: not for distribution.

ISBN: [ISBN TBD]

The specification-driven development methodology this book treats is Simon Martinelli's, which he calls the AI Unified Process, at `unifiedprocess.ai`. He presents it in "Lessons from Spec-driven Development," a talk given at AI Native DevCon in June 2026, at `youtu.be/odbNXv9xXjc`. Notes and References, at the back of this book, lists both alongside the standards and established practices the argument draws on.

Contents

PART 3: PRACTICE AND IMPLICATIONS

BACK MATTER

Chapter page numbers are not filled in for this draft edition. The folio printed at the foot of each page is the page number.

FRONT MATTER

Preface

You already know how to write code. This book is about what surrounds it: the decision the code is supposed to express, written down before the code exists to express it. Two examples run through the chapters ahead, a multi-location clinic group trying to let patients book appointments online, and a retail bank trying to move a core ledger off a COBOL codebase whose original designers have all left. Neither gets explained in full here. Chapter 1 opens with the clinic. Chapter 9 opens with the bank. This page only tells you they're coming.

By the last chapter, you'll be able to write a specification (a vision, a set of requirements, a use case, a domain model, and the architecture and skill conventions that surround them) precise enough that an AI agent implements it correctly on the first pass, before review has to find what a guess got wrong. You'll also be able to run the same discipline in reverse: reading a system that has no specification, and pulling a working specification out of its behavior, because in most legacy systems, behavior is the only documentation that survived.

Some of what follows will read like process, because it is process: writing a specification asks for hours before it hands any back. The next page sorts the chapters ahead into three reading paths. Pick the one that matches what you're actually deciding this week.

How to Read This Book

The three paths below are cuts through one argument, not three different books. The practitioner path is the whole book in order, and it is the right default. Take one of the other two when you have a specific decision in front of you and want the chapters that bear on it first. No path is a substitute for the chapters it skips; it is an order of arrival.

■ PRACTITIONER

Read Chapters 1 through 11 in order. This path gives you the complete methodology, front to back, for applying it to your own feature or team.

● ARCHITECT

Read the Foreword, then Chapters 2, 4, 6, 7, and 11, then Appendix C. This path runs from vision through architecture and skills and ends at the fully worked specification template, for a reader who owns consistency across teams rather than a single feature. Three of its stops lean on chapters it leaves out: Chapter 4 and Appendix C both work from the Schedule Appointment use case and the booking rules built in Chapter 3, and Chapter 11 argues from the legacy ledger recovered in Chapter 9. Read those two chapters first if either is unfamiliar.

◆ MANAGER

Read the Foreword, Chapter 1, Appendix A, Chapter 8, Chapter 10, and the Afterword. This path makes the case for the change and its effect on team structure and roles, without the detail of writing a specification yourself.